

LAPORAN PRATIKUM
ALGORITMA DAN PEMROGRAMAN
QUEUE (ANTRIAN)

Disusun Oleh:

Khairun Nisa

2511532015

Dosen Pengampu:

DR. WAHYUDI, S.T, M.T

Asisten Pratikum:

Sherly Sukmadira



FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas terselesaikannya laporan praktikum Algoritma dan Pemrograman ini. Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum yang telah dilaksanakan pada tanggal 27 April 2026. Melalui laporan ini, penulis berharap dapat memberikan gambaran yang jelas mengenai QUEUE (ANTRIAN).

Penulis mengucapkan terima kasih kepada Bapak Dosen serta asisten laboratorium yang telah membimbing selama praktikum berlangsung. Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari kata sempurna, baik dari segi penyusunan, bahasa, maupun kedalaman materi. Oleh karena itu, penulis dengan rendah hati menerima segala kritik dan saran yang membangun demi perbaikan dan penyempurnaan laporan di masa mendatang. Semoga laporan ini dapat bermanfaat bagi pembaca dan menjadi referensi untuk perkembangan ilmu pengetahuan di teknologi informasi.

Padang, 29 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Pratikum	1
1.3 Manfaat Pratikum	2
BAB II PEMBAHASAN	3
2.1 QueueArray.....	3
2.2 QueueArrayDriver	5
2.3 ReverseData	7
2.4 QueueLinkedList.....	8
2.5 IterasiQueue	10
BAB III KESIMPULAN	12
3.1 Kesimpulan	12
DAFTAR PUSTAKA	13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan komponen fundamental dalam pemrograman yang berperan penting dalam pengelolaan dan organisasi data. Salah satu struktur data linear yang sering digunakan dalam berbagai aplikasi adalah queue (antrian) yang menerapkan prinsip FIFO (First In First Out), di mana elemen yang pertama masuk akan menjadi elemen yang pertama keluar.

Struktur data queue adalah salah satu pilar bagi siapa saja yang ingin menjadi Software Engineering yang andal. Dalam dunia pemrograman yang kompleks, di mana ratusan ribu permintaan data harus dikelola setiap detik, memiliki mekanisme untuk mengatur aliran informasi secara tertib bukanlah sebuah kemewahan, melainkan kebutuhan mutlak. Struktur data queue adalah sebuah koleksi linear dari elemen-elemen data yang operasi penyisipan (insertion) dan penghapusan (deletion)-nya dilakukan pada dua ujung yang berbeda [1].

Queue dapat diimplementasikan menggunakan berbagai pendekatan struktur data, diantaranya menggunakan array dan linked list. Implementasi queue berbasis array menawarkan keunggulan dalam hal kecepatan akses dan penggunaan memori yang lebih efisien untuk ukuran data yang ukurannya sudah diketahui sebelumnya. Sementara itu, implementasi menggunakan linked list memberikan fleksibilitas lebih dalam pengelolaan memori dinamis, terutama ketika jumlah data tidak dapat diprediksi sebelumnya.

Selain operasi dasar seperti enqueue yang berfungsi untuk menambahkan elemen dan dequeue yang berfungsi untuk menghapus elemen, pemahaman tentang iterasi pada queue dan manipulasi data seperti membalikkan elemen atau reverse juga merupakan keterampilan penting yang harus dikuasai.

1.2 Tujuan Praktikum

1. Mahasiswa mampu memahami dan menjelaskan prinsip kerja struktur data queue dengan mekanisme FIFO.

2. Mahasiswa mampu mengimplementasikan operasi dasar queue yang meliputi enqueue, dequeue, peek, dan isEmpty.
3. Mahasiswa mampu mengembangkan algoritma untuk melakukan operasi lanjutan pada queue, seperti iterasi untuk menelusuri elemen dan pengembalian urutan/reverse tanpa merusak struktur data asli.
4. Mahasiswa mampu menerapkan struktur data queue untuk menyelesaikan studi kasus sederhana yang merepresentasikan permasalahan antrian dalam dunia nyata.

1.3 Manfaat Praktikum

1. Memperkuat pemahaman konseptual mengenai struktur data linear dan manajemen memori dalam ilmu komputer,
2. Melatih kemampuan berpikir algoritmik dan logika pemrograman melalui implementasi struktur data yang sistematis.
3. Memberikan fondasi yang kuat untuk mempelajari materi lanjutan berikutnya.

BAB II

PEMBAHASAN

2.1 Queue Array

```
package pekan4_2511532015;

public class QueueArray_2511532015 {
    int front, rear, size;
    int capacity;
    int array[];

    public QueueArray_2511532015 (int capacity) {
        this.capacity = capacity;
        front = this.size = 0;
        rear = capacity - 1;
        array = new int [this.capacity];
    }

    boolean isFull_2015 (QueueArray_2511532015 queue) {
        return (queue.size == queue.capacity);
    }

    boolean isEmpty_2015 (QueueArray_2511532015 queue) {
        return (queue.size == 0);
    }

    void enqueue_2015 (int item) {
        if (isFull_2015 (this))
            return;
        this.rear = (this.rear + 1)% this.capacity;
        this.array [this.rear]= item;
        this.size = this.size + 1;
        System.out.println (item + " enqueue to queue");
    }

    int dequeue_2015() {
        if (isEmpty_2015(this))
            return Integer.MIN_VALUE;
        int item = this.array[this.front];
        this.front = (this.front + 1)% this.capacity;
        this.size = this.size - 1;
        return item;
    }

    int front_2015() {
        if (isEmpty_2015 (this))
            return Integer.MIN_VALUE;

        return this.array[this.front];
    }

    int rear_2015() {
        if (isEmpty_2015(this))
            return Integer.MIN_VALUE;
        return this.array[this.rear];
    }
}

//mencetak elemen antrean
```

```

void display_2015() {
    int i;
    if (front == rear) {
        System.out.printf("\nAntrian Kosong\n");
        return;
    }
    //kunjungi dari belakang dan cetak
    for (i = front; i < rear; i++) {
        System.out.printf(" %d <--", array[i]);
    }
    return;
}
}

```

Kode Program ini adalah cetak biru (blueprint) untuk membuat tempat penyimpanan data yang bekerja seperti antrian, di mana data yang datang pertama akan dilayani pertama sesuai dengan prinsip queue yaitu FIFO. Di sini tempat penyimpanannya menggunakan Array.

2.1.1 Penginisialisasi variable

- Capacity: Batas maksimal orang yang boleh mengantri
- Array[]: sebagai tempat untuk menyimpan data atau nomor antrian
- Front: paling depan akan dilayani
- Rear: menunjuk posisi kosong terakhir untuk orang baru masuk
- Size: menghitung ada berapa orang yang sedang antri

2.1.2 QueueArray_2015(int capacity) merupakan konstruktor yang membuat objek baru dan menyiapkan array sesuai kapasitas dan mengatur posisi awal front dan size menjadi 0

2.1.3 Fungsi pengecekan

- boolean isFull_2015(QueueArray_2015 queue): bandingkan size dengan capacity, jika size == capacity, maka return true. Jika tidak, return false
- boolean isEmpty_2015(QueueArray_2015 queue): cek apakah size == 0, jika ya maka return true

2.1.4 Fungsi Operasi Pertama (Main)

- void enqueue_2015(int item): Menambahkan data, dimana dipanggil dulu isFull apabila full maka return berhenti, jika

tidak full maka, $\text{rear} = (\text{rear} + 1) \% \text{capacity}$ dan simpan item di poisis `array[rear]` serta size nya bertambah 1

- `void dequeue_2015()`: Menghapus data

2.1.5 `front_2015()`: berfungsi untuk melihat elemen paling depan

2.1.6 `rear_2015()`: berfungsi untuk melihat elemen paling belakang

2.1.7 Fungsi untuk menampilkan yaitu `display_2015()` yang berfungsi untuk mencetak semua elemen

Implementasi queue berbasis array pada praktikum ini berhasil menerapkan prinsip FIFO melalui mekanisme circular queue yang efisien. Dengan pengendalian penunjuk front dan rear serta operasi modulo (%), struktur ini mampu memanfaatkan memori secara optimal dan menghindari pemborosan ruang. Seluruh operasi dasar seperti enqueue, dequeue, isEmpty, dan isFull telah dilengkapi dengan validasi keamanan untuk mencegah kondisi overflow maupun underflow.

2.2 QueueArrayDriver

```
package pekan4_2511532015;

public class QueueArrayDriver_2511532015 {

    public static void main(String[] args) {
        QueueArray_2511532015 queue_2015 = new
        QueueArray_2511532015 (1000);
        queue_2015.enqueue_2015(10);
        queue_2015.enqueue_2015(20);
        queue_2015.enqueue_2015(30);
        queue_2015.enqueue_2015(40);
        System.out.println ("Item di depan "+
        queue_2015.front_2015());
        System.out.println ("Item paling belakang "+
        queue_2015.rear_2015());
        System.out.println ("tampilkan queue");
        queue_2015.display_2015();
        System.out.println ();
        System.out.println (queue_2015.dequeue_2015() + " dihapus
        dari queue");
        System.out.println ("Item di depan: "+
        queue_2015.front_2015());
        System.out.println ("Item dibelakang: "+
        queue_2015.rear_2015());
        System.out.println ("Tampilan queue setelah daru data
        dihapus");
        queue_2015.display_2015();
    }
}
```



```
}
```

Program ini berfungsi sebagai pengujian yang isinya hanya main method yang bertindak sebagai simulator. Tugasnya hanya memanggil, menguji, dan menampilkan hasil dari fungsi-fungsi yang sudah dibuat di program QueueArray.

2.2.1 QueueArray_2511532015 queue_2015 = new QueueArray_2511532015(1000): program ini membuat objek antrian dengan kapasitas maksimla 1000 slot dengan kondisi awal anriannya kosong.

2.2.2 Angka 10, 20, 30, dan 40 dimasukkan satu per sat uke belakang antiran dengan memanggil fungsi enqueue. Sesuai urutan FIFO, urutan di memorinya [10, 20, 30, 40].

2.2.3 Program memanggil fungsi front_2015() utnuk mengembalikan nilai paling depan (10) dan fungsi rear_2015() untuk mengembalikan nilai paling belakang (40)

2.2.4 Memanggil fungsi display untuk mencetak semua elemen yang sedang mengantre dari depan ke belakang

2.2.5 Fungsi dequeue_2015() dipanggil untuk menghapus angka paling awal masuk yaitu 10, sehingga antrian tinggal [20, 30, 40]

2.2.6 Program kembali memanggil front, rear, dan display untuk membuktikan bahwa setelah penghapusan, antrian telah bergeser sesuai prinsip FIFO (sekarang depan jadi 20, belakang tetap 40)

2.2.7 Outputnya akan seperti:

```
ue. @ Javadoc Console X Progress Git Staging History
<terminated> QueueArrayDriver_2511532015 [Java Application] C:\Users\US
10 enqueue to queue
20 enqueue to queue
30 enqueue to queue
40 enqueue to queue
Item di depan 10
Item paling belakang 40
tampilkan queue
10 <-- 20 <-- 30 <--
10 dihapus dari queue
Item di depan: 20
Item dibelakang: 40
Tampilan queue setelah daru data dihapus
20 <-- 30 <--
```

Gambar 2.2.1: Hasil Output program

QueueArrayDriver_2511532015

2.3 ReverseData

```
package pekan4_2511532015;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511532015 {

    public static void main(String[] args) {
        Queue<Integer> q_2015 = new LinkedList<Integer>();
        q_2015.add(1);
        q_2015.add(2);
        q_2015.add(3); // [1, 2, 3]
        System.out.println ("Sebelum reverse = " + q_2015);
        Stack<Integer> s = new Stack<Integer>();
        while (!q_2015.isEmpty()){ // Q -> S
            s.push (q_2015.remove());
        }
        while (!s.isEmpty()) { // S -> Q
            q_2015.add(s.pop());
        }
        System.out.println ("Sesudah reverse = " + q_2015); // [3,
        2, 1]
    }
}
```

Program ini membalikkan data pada struktur queue yang secara alami bersifat FIFO dengan memanfaatkan sifat LIFO dari struktur Stack.

2.3.1 Pertama mengisi queue dengan method add yang menambahkan angka 1, 2, dan 3. Kemudian mencetak untuk perbandingan sebelum reverse

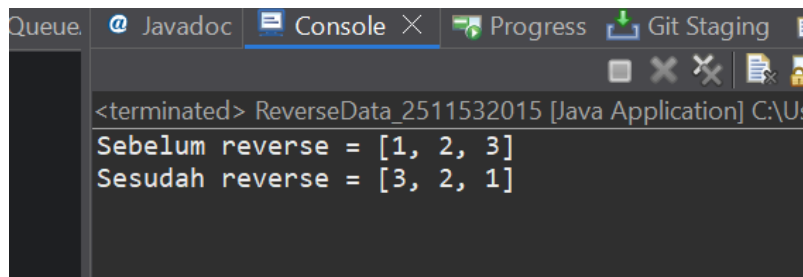
2.3.2 Kemudian program pindah dari queue ke stack untuk reverse, karena stack bersifat LIFO, maka elemen yang terakhir masuk (3) akan menjadi yang paling depan sekarang.

- Langkah 1: Queue ambil 1 → Stack push 1
Queue: [2, 3] Stack: [1] (1 di bawah)
- Langkah 2: Queue ambil 2 → Stack push 2
Queue: [3] Stack: [1, 2] (2 di atas 1)
- Langkah 3: Queue ambil 3 → Stack push 3
Queue: [] Stack: [1, 2, 3] (3 di PALING ATAS)

2.3.3 Kemudian pindah lagi dari stack ke queue untuk hasil akhirnya

- Langkah 1: Stack pop 3 → Queue add 3
Stack: [1, 2] Queue: [3]
- Langkah 2: Stack pop 2 → Queue add 2
Stack: [1] Queue: [3, 2]
- Langkah 3: Stack pop 1 → Queue add 1
Stack: [] Queue: [3, 2, 1]

2.3.4 Terakhir menampilkan hasilnya, ketika dijalankan program akan menghasilkan output:



Gambar 2.3.1: Hasil Output program ReverseData_2511532015

2.4 QueueLinkedList

```
package pekan4_2511532015;
```

```

import java.util.LinkedList;
import java.util.Queue;

public class QueueLinkedList_2511532015 {

    public static void main(String[] args) {
        Queue<Integer> q_2015 = new LinkedList <>();
        //tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
        for (int i = 0; i < 6; i++)
            q_2015.add(i);

        //menampilkan usu antrian
        System.out.println ("Elemen antrian "+ q_2015);

        //untuk menghapus kepala antrian
        int hapus = q_2015.remove();
        System.out.println ("Hapus elemen = " + hapus);
        System.out.println (q_2015);

        //untuk melihat antrian terdepan
        int depan = q_2015.peek();
        System.out.println ("Kepala antrian = "+ depan);

        int banyak = q_2015.size();
        System.out.println ("Size antrian = "+ banyak);

    }
}

```

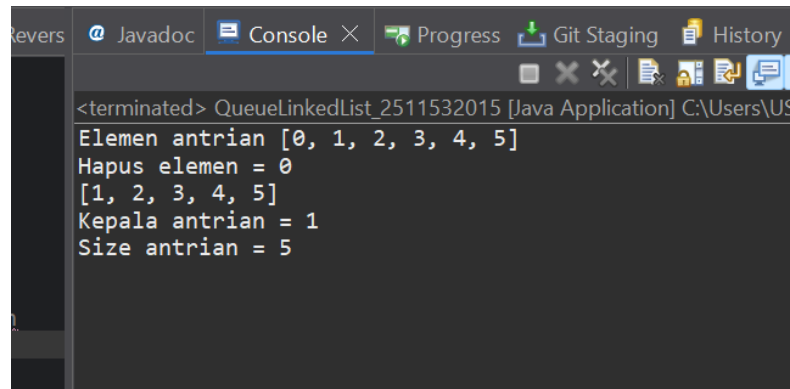
Program ini adalah implementasi sederhana dari struktru queue menggunakan fasilitas bawaan java, yang bertujuan untuk melihat langsung bagaimana prinsip FIFO bekerja dalam kode nyata.

- 2.4.1 Program menyiapkan antrian kosong yang siap menampung data integer. Karena pakai LinkedList, maka ukurannya dinamis bisa bertamnah sesuai kebutuhan, tidak perlu netapin kapasitas awal seperti array
- 2.4.2 Kemudian program mengisi antrian dengan perulangan. for (int i = 0; i < 6; i++): perulangan ini menambahkan angka 0, 1, 2, 3, 4, 5 ke dalam antrian
- 2.4.3 Untuk menghapus elemen paling depan (Dequeue) digunakan method remove() yang nantinya akan mengambil dan menghapus elemen paling depan, sesuai prinsip FIFO angka 0 yang akan dimabil

2.4.4 Melihat antrian paling depan menggunakan method `peek()` yaitu angka 1

2.4.5 Menghitung berapa elemen yang masih ada di dalam antrian menggunakan method `size()`, karena 0 sudah di remove maka antrian sisa 5 elemen

2.4.6 Ketika dijalankan program akan menghasilkan output:



Gambar 2.4.1: Hasil output program

QueueLinkedList_2511532015

2.5 IterasiQueue

```
package pekan4_2511532015;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;

public class IterasiQueue_2511532015 {

    public static void main(String[] args) {
        Queue<String> q_2015 = new LinkedList<>();

        q_2015.add("Praktikum");
        q_2015.add("Struktur");
        q_2015.add("Data");
        q_2015.add("Dan");
        q_2015.add("Algoritma");
        Iterator<String> iterator = q_2015.iterator();
        while (iterator.hasNext()) {
            System.out.print (iterator.next() + " ");
        }
    }
}
```

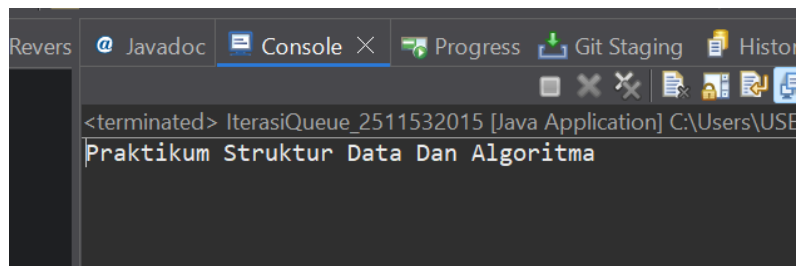
Program ini mendemonstrasikan teknik traversal/iterasi pada struktur data Queue. Berbeda dengan dequeue() yang sekaligus mengambil dan menghapus elemen, iterasi hanya membaca elemen dari depan ke belakang sesuai urutan FIFO, lalu membiarkan antrian tetap utuh.

2.5.1 Program membuat antrian kosong yang khusus menyimpan string/teks

2.5.2 Kemudian menambahkan kata dengan method add()

2.5.3 iterator() adalah metode yang meminta Java memberikan "kursor penunjuk" ke awal antrian.

2.5.4 Program akan menghasilkan output:



Gambar 2.5.1: Hasil output program IterasiQueue_2511532015

BAB III

KESIMPULAN

3.1 Kesimpulan

Praktikum ini berhasil mengimplementasikan struktur data Queue berbasis prinsip FIFO melalui dua pendekatan utama: antrian sirkular menggunakan array dan antrian dinamis menggunakan LinkedList Java. Melalui serangkaian pengujian, telah dikuasai operasi fundamental (enqueue, dequeue, peek, size), teknik iterasi aman tanpa menghapus data, serta manipulasi lanjutan seperti pembalikan urutan antrian dengan bantuan Stack. Perbandingan implementasi membuktikan bahwa array menawarkan efisiensi memori statis dengan logika $\% \text{ capacity}$, sementara LinkedList memberikan fleksibilitas dinamis tanpa perlu manajemen indeks manual. Secara keseluruhan, praktikum ini tidak hanya memperkuat pemahaman konseptual Queue, tetapi juga melatih kemampuan analisis efisiensi, validasi kondisi batas (isEmpty/isFull), dan penerapan struktur data dalam konteks pemrograman nyata seperti penjadwalan, buffering, dan manajemen aliran data.

DAFTAR PUSTAKA

- [1] Prasatya, "Apa itu Struktur Data Queue? Pengertian, Jenis, dan Kegunaannya!," KelasFullStack, 29 October 2025. [Online]. Available: <https://www.kelasfullstack.id/blog/apa-itu-struktur-data-queue-pengertian-jenis-dan-kegunaannya>. [Accessed 28 April 2026].

PENJELASAN TUGAS PRAKTIKUM PEKAN 4

1. Kode program AntrianLoket_2511532015

```
import java.util.Stack;
public class AntrianLoket_2511532015 {
    int front, rear, size;
    int capacity;
    int max;
    String[] queue;

    public AntrianLoket_2511532015 (int capacity) {
        max = capacity;
        queue = new String [max];
        front = this.size = 0;
        rear = capacity - 1;
    }
    boolean isFull_2015 () {
        return (size == max);
    }
    package pekan4_2511532015;

    boolean isEmpty_2015 () {
        return (size == 0);
    }
    //method enqueue = menambahkan data
    public void enqueue_2015 (String pelanggan) {
        if (isFull_2015()) {
            System.out.println ("Antrian penuh");
            return;
        }
        rear = (rear + 1) % max;
        queue [rear] = pelanggan;
        size = size + 1;
    }
    //method dequeue = menghapus data dari depan
    String dequeue_2015() {
        if (isEmpty_2015()) {
            System.out.println ("Antrian Kosong");
            return null;
        }
        String pelanggan = queue [front];
        front = (front + 1) % max;
        size = size - 1;
        return pelanggan;
    }
    //method display = menampilkan antrian
    void display_2015() {
        int i= front;
        if (isEmpty_2015()) {
            System.out.println ("Antrian Kosong");
            return ;
        }
        for (int j = 0; j < size; j++) {
            System.out.println((j+1) + ". " + queue [i]);
            i = (i+1)% max;
        }
    }
}
```

```

        return;
    }
    //method reverse
    void reverse_2015() {
        Stack<String> s = new Stack<>();

        while (!isEmpty_2015()) {
            String data = dequeue_2015();
            s.push(data);
        }
        while (!s.isEmpty()) {
            String data = s.pop();
            enqueue_2015(data);
        }
    }
}

```

Program ini merancang struktur data Queue (Antrian) berbasis array dengan prinsip Circular FIFO (First In First Out) yang mengimplementasikan sistem antrian loket pelayanan. Class `AntrianLoket_2511532015` berfungsi sebagai blueprint yang mendefinisikan seluruh logika dan operasi fundamental antrian, seperti deklarasi komponen antrian, operasi dasar (enqueue, dequeue, display), validasi kondisi yang meliputi method `isEmpty` dan `isFull`, serta manipulasi berupa pembalikan urutan antrian atau reverse.

2. Kode Program `AntrianLoketDriver_2511532015`

```

package pekan4_2511532015;

import java.util.Scanner;

public class AntrianLoketDriver_2511532015 {

    public static void main(String[] args) {
        Scanner input = new Scanner (System.in);
        AntrianLoket_2511532015 antrian = new
        AntrianLoket_2511532015(100);
        int pilih;

        do {
            System.out.println ("\n=== PROGRAM ANTRIAN LOKET
            ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            pilih = input.nextInt();
            input.nextLine();

```

```

        switch (pilih) {
        case 1:
            System.out.print("Masukkan nama pelanggan: ");
            antrian.enqueue_2015(input.nextLine());
            System.out.println ("Data berhasil ditambahkan ke
antrian");
            break;
        case 2:
            String terlayani = antrian.dequeue_2015();
            if (terlayani != null)
                System.out.println(terlayani + " telah dilayani");
            break;
        case 3:
            System.out.println ("Isi antrian:");
            antrian.display_2015();
            break;
        case 4:
            System.out.println ("Isi antrian:");
            antrian.reverse_2015();
            antrian.display_2015();
            break;
        case 5:
            System.out.println("Program selesai. Terima
kasih!");
            break;
        default:
            System.out.println("Pilihan tidak valid.");
        }
    } while (pilih != 5);
    input.close();
}
}

```

Kelas ini berfungsi sebagai pusat kendali dan antarmuka interaktif yang menghubungkan pengguna dengan logika antrian yang telah dirancang di kelas blueprint. Tanpa kelas ini, program hanya berupa kumpulan method yang tidak dapat dijalankan secara langsung.

```
mas  
@ Javadoc Search Console X Progress Git Staging History  
AntrianLoketDriver_2511532015 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj  
  
=== PROGRAM ANTRIAN LOKET ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 1  
Masukkan nama pelanggan: Khairun nisa  
Data berhasil ditambahkan ke antrian  
  
=== PROGRAM ANTRIAN LOKET ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 1  
Masukkan nama pelanggan: Cordelia  
Data berhasil ditambahkan ke antrian  
  
=== PROGRAM ANTRIAN LOKET ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu:
```

Gambar 1.1: Output untuk menambahkan data

```
5.java  
5.java  
java  
=== PROGRAM ANTRIAN LOKET ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 3  
Isi antrian:  
1. Khairun nisa  
2. Cordelia  
  
=== PROGRAM ANTRIAN LOKET ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 4  
Isi antrian:  
1. Cordelia  
2. Khairun nisa
```

Gambar 2.2: Output untuk menampilkan data dan reverse data

```
=== PROGRAM ANTRIAN LOKET ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 2  
Cordelia telah dilayani
```

Gambar 2.3: Output untuk menghapus data (telah dilayani)

```
=== PROGRAM ANTRIAN LOKET ===  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih menu: 5  
Program selesai. Terima kasih!
```

Gambar 2.4: Output keluar program